TaskFlow — Roteiro de Apresentação Alinhado

Atividade 1: Lab de Esteira Segura | Grupo Room 4 · Repositório: Shuralot/DevSecOps-Room4 · Duração: 10 minutos cronometrados

Estrutura da Apresentação (Condução Integral)

Apresentador Principal	Slides do PDF	Foco Temático	Tempo Estimado
Júlio Santos (@Shuralot)	01 ao 08 (Integral)	Abertura, Pipelines CI/CD, Correções no app.py, Hardening Docker, Demo do Gate e Conclusão	0:00–10:00 (10 min)
Diego Buarque, Filipe Almeida e Equipe	Coautores	Desenvolvimento conjunto das esteiras e suporte técnico nas arguições da banca	Suporte / Q&A;

Checklist Operacional Pré-Apresentação (Obrigatório)

1. Visibilidade & Colaboradores: Repositório privado com @HardSource como colaborador com permissão de leitura.
2. Proteção da Branch main: PR obrigatório, aprovação do CODEOWNERS e 5 checks obrigatórios ativados (CI - build lint testes, SAST - Semgrep, Segredos - Gitleaks, SCA - pip-audit, Container - build e Trivy).
3. Ambiente Preparado: Deixar abas abertas no navegador: PR da falha controlada, logs do GitHub Actions, código app.py e docs/hardening.md.

Roteiro Slide a Slide (Alinhado ao PDF de Apresentação)

SLIDE 01 | CONTEXTO (0:00–0:30) **Responsável: Júlio Santos**

Tema: TaskFlow — Esteira Segura de CI

O que mostrar em tela: Slide 1 do PDF ou página inicial do repositório no GitHub (Shuralot/DevSecOps-Room4).

Fala Sugerida (Script):
"Boa noite professor. Somos o Grupo Room 4. Nosso objetivo foi transformar a TaskFlow em uma aplicação com segurança embutida em uma esteira de CI no GitHub Actions. Nosso foco não foi apenas rodar scanners, mas construir quality gates de verdade, onde o Pull Request é o portão de controle e vulnerabilidades bloqueiam o merge na main. Dividimos a apresentação entre os membros reais da equipe para demonstrar código, pipelines, containers e a demonstração prática do gate."

Pontos Técnicos / Decisões: Destacar que o repositório é privado e o professor @HardSource está adicionado como leitor.

SLIDE 02 | ESTEIRA (0:30–1:30) **Responsável: Júlio Santos**

Tema: Verificações antes da Integração

O que mostrar em tela: Pasta .github/workflows/ (ci.yml, sast.yml, sca.yml, container.yml) e .github/CODEOWNERS.

Fala Sugerida (Script):
"Estruturamos a esteira em 4 workflows independentes e modulares: CI com checagem de hashes e lint, SAST com Semgrep e Gitleaks, SCA com pip-audit e Dependabot, e Container com build e Trivy. Em todos os pipelines aplicamos o princípio do privilégio mínimo com permissions: contents: read e fixamos todas as GitHub Actions por hash SHA completo para neutralizar ataques na cadeia de suprimentos. A branch main exige a aprovação de todos esses 5 checks para liberar qualquer merge."

Pontos Técnicos / Decisões: Explicar que pipeline que falha não adianta se a main não tiver proteção ativa obrigando os checks.

SLIDE 03 | CÓDIGO (1:30–2:30) Responsável: Júlio Santos

Tema: Correções Exigidas na Aplicação

O que mostrar em tela: Arquivo app.py (funções create_app, create-user, login, tasks e busca).

Fala Sugerida (Script):
"Em vez de mascarar regras dos scanners, corrigimos o código na fonte: no app.py, substituímos todas as consultas concatenadas por consultas SQL parametrizadas com placeholders ?, impedindo que a entrada do usuário modifique a instrução do banco. Eliminamos senhas em texto puro usando hashing com salt (generate_password_hash e check_password_hash da Werkzeug). E a SECRET_KEY foi completamente retirada do repositório: ela é lida estritamente de TASKFLOW_SECRET_KEY do ambiente, falhando imediatamente se não for definida."

Pontos Técnicos / Decisões: Mencionar também a inclusão de proteção CSRF com CSRFProtect e validação de senhas com mínimo de 12 caracteres.

SLIDE 04 | DOCKER (2:30–3:30) Responsável: Júlio Santos

Tema: Hardening do Container

O que mostrar em tela: Arquivo Dockerfile e script scripts/docker-run-seguro.sh.

Fala Sugerida (Script):
"Corrigimos as 6 falhas críticas da imagem original: 1) Trocamos a base genérica latest por Python 3.12 Alpine fixada por versão e digest SHA256; 2) Removemos a execução como root, criando usuário dedicado 10001:10001; 3) Removemos ADMIN_PASSWORD da imagem; 4) Substituímos o servidor Flask de desenvolvimento pelo Gunicorn de produção; 5) Desligamos o modo debug; 6) Travamos dependências por hashes. No runtime, nosso docker-run-seguro.sh roda o container em read-only, com cap-drop=ALL e no-new-privileges."

Pontos Técnicos / Decisões: Destacar a escolha por Alpine para reduzir a superfície de ataque e o apk upgrade para atualizar pacotes do SO.

SLIDE 05 | EVIDÊNCIAS LOCAIS (3:30–4:30) Responsável: Júlio Santos

Tema: Resultados Registrados

O que mostrar em tela: Arquivo docs/hardening.md, docs/evidencias/resumo.json e .gitleaksignore.

Fala Sugerida (Script):
"Registramos com transparência as métricas antes e depois: no Semgrep local com as regras p/python, p/flask e p/security-audit, tínhamos 10 achados no código original e zero no código corrigido. No scan de imagem com o Trivy para severidades HIGH e CRITICAL, caímos de 388 ocorrências na imagem base original para zero na imagem final. No Gitleaks com fetch-depth: 0, detectamos chaves públicas GPG da base oficial Python; em vez de desligar a regra, tratamos no .gitleaksignore pontualmente pelos fingerprints exatos, mantendo o gate ativo."

Pontos Técnicos / Decisões: Ressaltar que zero HIGH/CRITICAL é o retrato do scan na data e não uma garantia absoluta perpétua; por isso Dependabot e scans contínuos são vitais.

SLIDE 06 | EXECUTAR NO GITHUB (4:30–6:30) Responsável: Júlio Santos

Tema: PR com Vulnerabilidade Controlada (Demo ao Vivo)

O que mostrar em tela: Aba Pull Requests no GitHub, log do Semgrep no GitHub Actions e botão de Merge bloqueado.

Fala Sugerida (Script):
"Demonstramos agora o shift-left na prática: abrimos este Pull Request de teste contendo uma consulta SQL concatenada com dados de formulário no login. Ao disparar o pipeline, o job do Semgrep executou e a flag --error fez a esteira falhar com exit-code 1, indicando a vulnerabilidade exata no log. Voltando à tela do Pull Request, vejam que a branch protection da main impediu completamente o merge. A esteira atuou como barreira bloqueante antes que a vulnerabilidade chegasse à produção."

Pontos Técnicos / Decisões: Enfatizar a diferença entre o pipeline falhar (Semgrep reprova) e o merge ser bloqueado (GitHub branch protection).

SLIDE 07 | EXECUTAR NO GITHUB (6:30–8:30) Responsável: Júlio Santos

Tema: Correção no Mesmo PR

O que mostrar em tela: Commit corretivo no mesmo PR, execução verde do Actions e botão de Merge liberado.

Fala Sugerida (Script):
"No mesmo Pull Request, submetemos o commit corretivo restaurando a consulta parametrizada com placeholders. O GitHub Actions executou automaticamente a suíte de validações. O Semgrep rodou e foi aprovado com zero achados. Com os 5 required checks verdes e a validação do CODEOWNERS aprovada, a proteção de branch do GitHub desbloqueou o botão de Merge. O desenvolvedor corrigiu o problema sem desativar a segurança, comprovando a eficácia do Quality Gate."

Pontos Técnicos / Decisões: Mostrar que nenhum scan foi pulado e nenhuma flag de bypass foi utilizada.

SLIDE 08 | ENCERRAMENTO (8:30–10:00) Responsável: Júlio Santos

Tema: Aprendizado e Conferência Final

O que mostrar em tela: Slide 8 do PDF, arquivo docs/resumo-atividade.md e tabela de status checks.

Fala Sugerida (Script):
"Para encerrar, nosso aprendizado mais importante foi que ferramentas isoladas não resolvem segurança: a proteção real decorre da disciplina de governança da branch main exigindo os quality gates. Aprendemos também a importância da transparência: corrigir falhas em vez de usar supressões artificiais como # nosemgrep ou --exit-zero. Cumprimos todos os requisitos do enunciado: repositório privado com professor leitor, branch protection ativa, código corrigido, container seguro e evidências reprodutíveis. Estamos à disposição da banca."

Pontos Técnicos / Decisões: Todos os membros ficam a postos para responder perguntas da banca.

Guia Rápido para Perguntas da Banca / Professor

Pergunta Provável	Resposta Rápida e Firme
O que realmente impede o merge?	A regra de proteção de branch do GitHub (Required Status Checks). O pipeline falha via exit-code, e o GitHub não habilita o merge sem aprovação de todos os 5 checks.
Por que fetch-depth: 0 no Gitleaks?	Para analisar o histórico completo do Git. O checkout padrão baixa só o último commit; apagar um segredo de um arquivo não o apaga dos commits anteriores.
Para que serve --require-hashes no pip?	Garante integridade e combate ataques de Supply Chain: o hash SHA256 do pacote baixado deve bater exatamente com o lockfile pré-auditado.
Zero no Trivy significa 100% seguro?	Não. Significa zero CVEs HIGH/CRITICAL conhecidas na data do scan. Como novas vulnerabilidades surgem continuamente, Dependabot e rebuilds regulares são essenciais.